HashiCorp Certified: Terraform Associate（004）　目標 4f・4g　#09

依存関係とカスタム条件

暗黙の依存と depends_on、依存グラフと terraform graph、lifecycle（create_before_destroy・prevent_destroy・ignore_changes・replace_triggered_by）と terraform_data、変数の validation・precondition・postcondition・check ブロック、検証の順番、継続的検証

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の depends_on meta-argument reference・lifecycle meta-argument reference・Dependency Graph・Command: graph・terraform_data resource reference・Validate your configuration・check block reference、HCP Terraform の Health assessments）を書いた。英語の原文は『』で引用した

第1章

依存関係

ふつうは参照で。depends_on は最後の手段。

暗黙の依存と depends_on

```
resource "aws_iam_instance_profile" "example" {
  role = aws_iam_role.example.name   # 参照 → 暗黙の依存
}
resource "aws_instance" "example" {
  ami                  = "ami-a1b2c3d4"
  instance_type        = "t2.micro"
  iam_instance_profile = aws_iam_instance_profile.example
  # 起動時に S3 を使う → ポリシーへの依存は参照に出てこない
  depends_on = [
    aws_iam_role_policy.example
  ]
}
```

暗黙の依存関係
引数がほかのリソースを参照 → 自動で順番が決まる（推奨）

depends_on（明示）
相手のふるまいに頼るが、データは使っていないとき

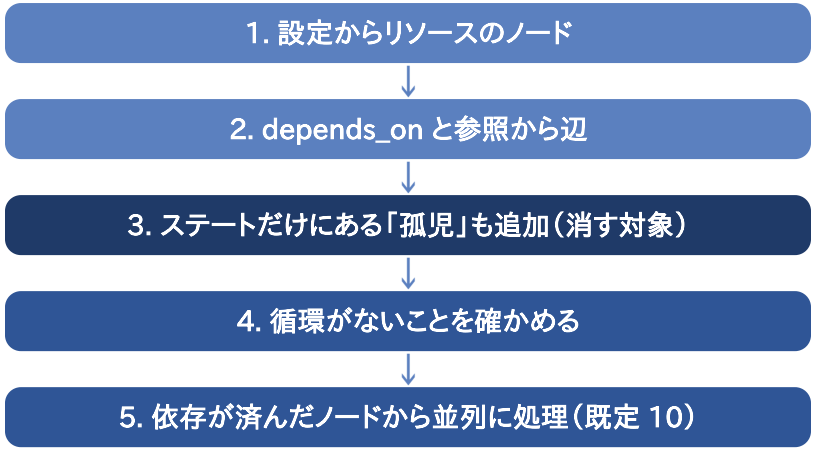
値は同じモジュールのリソース・子モジュールへの参照のリスト。任意の式は不可。

書けるブロック：resource・data・module・output・check・ephemeral。

『You should only use depends_on as a last resort』：保守的な計画になり、known after apply や余計な置きかえが増える（module に付けると特に）。

出典：depends_on meta-argument reference

依存グラフと terraform graph




```
$ terraform graph                  # resource・data の順番だけ
$ terraform graph -type=plan | dot -Tpng >graph.png
$ terraform graph -type=apply -draw-cycles
$ terraform apply -parallelism=5   # 同時に処理する数
```

出力は DOT 言語のテキスト。画像は Graphviz の dot で作る。

-type は plan・plan-refresh-only・plan-destroy・apply。

-parallelism はふつう変えない。API のレート制限はプロバイダー側のリトライで対処する。

出典：Dependency Graph／Command: graph

第2章

lifecycle のルール

作る・消す・置きかえるときのふるまいを変える。

4 つのルール

ルール	何をするか	覚えどころ
create_before_destroy	置きかえで「作ってから消す」	既定は消してから作る。opt-in（名前の衝突に注意）。依存先にも伝わり false にできない
prevent_destroy	消す計画をエラーに	DB を守る。destroy もできない。ブロックごと消すと守れない
ignore_changes	指定した属性の更新を無視	作るときは使う。all ＝更新しない。メタ引数は不可
replace_triggered_by	参照先が変わったら置きかえ	管理リソースのアドレスだけ。変数は terraform_data 経由

```
resource "aws_appautoscaling_target" "ecs_target" {
  lifecycle {
    create_before_destroy = true
    ignore_changes        = [tags]
    replace_triggered_by  = [aws_ecs_service.svc.id]
  }
}
```

『only literal values can be used』：lifecycle の値はリテラルだけ（変数・式は不可）。

出典：lifecycle meta-argument reference

terraform_data と -replace

```
variable "revision" {
  default = 1
}
resource "terraform_data" "replacement" {
  input = var.revision
}
resource "example_database" "test" {
  lifecycle {
    replace_triggered_by = [terraform_data.replacement]
  }
}
```

> terraform_data はプロバイダー不要（terraform.io/builtin/terraform）。input が変わると変更を計画する。

やりたいこと	使うもの
一度だけ作り直す	terraform apply -replace=ADDR（#05）
ある変更のたびに作り直す	replace_triggered_by
変数の変化で作り直す	terraform_data の input ＋ replace_triggered_by
ステートから外すだけ	removed ブロックの lifecycle { destroy = false }（#13）

出典：terraform_data resource reference／lifecycle meta-argument reference

第3章

カスタム条件

止める検証と、止めない検証。

4 つの検証と順番

順	検証	書く場所	いつ	失敗したら
1	validation	variable	計画の前（すぐ）	エラーで止まる
2	precondition	resource・data・ephemeral の lifecycle、output	計画のあと、作る前	エラーで止まる
3	postcondition	resource・data・ephemeral の lifecycle	計画・適用のあと	下流を止める（済んだことは戻さない）
4	check の assert	トップレベルの check	plan・apply の最後	警告だけで続ける

どれも condition と error_message が必須。error_message は文字列になる式なら何でも（テンプレート・ヒアドキュメント）。

値が apply まで分からなければ（EC2 のルートボリューム ID など）、apply のときに評価する。

出典：Validate your configuration（Order of validation）

precondition と postcondition

```
resource "aws_instance" "example" {
  instance_type = "t3.micro"
  ami           = data.aws_ami.example.id
  lifecycle {
    precondition {      # 前提：作る前
      condition     = data.aws_ami.example.architecture == "x86_64"
      error_message = "The selected AMI must be for the x86_64 architecture."
    }
    postcondition {     # 保証：作ったあと
      condition     = self.public_dns != ""
      error_message = "EC2 instance must be in a VPC that has public DNS hostnames enabled."
    }
  }
}
```

precondition = 前提（assumption）
プロバイダーの引数エラーより先に出る。count.index・each.key も使える

postcondition = 保証（guarantee）
self で自分の属性を参照（名前で参照すると循環）

output には precondition だけ書ける（外に出す・ステートに保存する前）。

依存先が多いなら、そのリソースに postcondition を 1 つ書くほうが分かりやすいことも。

出典：lifecycle meta-argument reference／Validate your configuration

check ブロックと継続的検証

```
check "health_check" {
  data "http" "terraform_io" {      # スコープ付き
    url = "https://www.terraform.io"
  }
  assert {                          # 1 つ以上必須
    condition     = data.http.terraform_io.status_code == 200
    error_message = "${data.http.terraform_io.url} returned an unhealthy status code"
  }
}
```

中の data はその check の中だけで参照でき、最後に読む。プロバイダーのエラーは警告に。

『The check block is the only validation that does not block operations』

最初の apply でうその警告が出るなら、中の data に depends_on（ロードバランサーなど）。data がそのリソースを直接参照していると効かない。

HCP Terraform のヘルス評価
ドリフト検出＋継続的検証（check・precondition・postcondition を定期的に評価）

出典：check block reference／depends_on meta-argument reference／Health assessments

本番での問われ方

ねらい	見るところ
依存関係	参照で暗黙の依存、depends_on はかくれた依存だけ・最後の手段、書けるブロック
グラフ	並列 10、-parallelism、terraform graph は DOT
lifecycle	4 つのルールの使い分け、値はリテラルだけ、変数は terraform_data 経由
カスタム条件	前提＝precondition・保証＝postcondition、self、output には precondition
check	止めない唯一の検証、スコープ付きデータ、継続的検証

出典：Exam Content List（004）4f・4g／本資料の問題演習（全 30 問）